Module 3
Pointers

3.1 Pointer Fundamentals

- A pointer is a variable that stores the address of another variable.
- Instead of holding a direct value, a pointer "points" to the location in memory where the actual data is stored.

3.3.1 Declaration

- A pointer variable is declared by using an asterisk (*) before the variable name, indicating that it will store an address.
- The data type of the pointer must match the data type of the variable it will point to.
- Syntax:

  datatype *pointer_name;
- Examples:

```
int *p;      // Declares a pointer ‘p’ that stores the address of an integer variable
float *q;    // Declares a pointer ‘q’ that stores the address of a float variable
char *r;     // Declares a pointer ‘r’ that stores the address of a character variable
```

3.3.2 Initialisation

- A pointer variable is initialised with the address of an existing variable.
- Example:

```
int a = 10;
int *p = &a;  // p now points to a
```

- Initialisation can also be done in two steps as follows:

```
int num = 26;
int *ptr;   // declaration
ptr = &num;   // initialization later
```

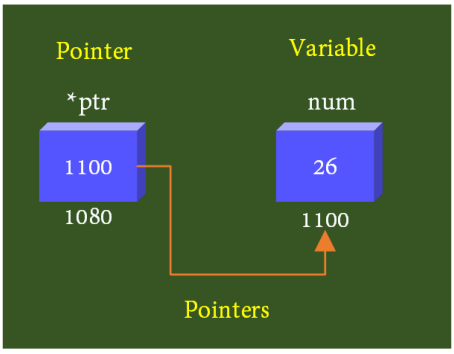


Fig. 3.1 Pointer concept

3.1.3 Address-of (&) and Value-at (*) Operators

- Address-of Operator (&): This operator is used to get the memory address of a variable.

- Dereference Operator (*): Also known as the indirection operator, is used to access the value stored at the memory address held by a pointer.
- Example:

```
void main() {
   int a = 5;
   int *p;
   p = &a;   // p stores address of a
   printf("Address of a = %p\n", &a); // Prints address of a
   printf("Address stored in p = %p\n", p); // Prints address of a
   printf("Value of a = %d\n", a);  //Prints 5
   printf("Value using pointer = %d\n", *p);  //Prints 5
}
```

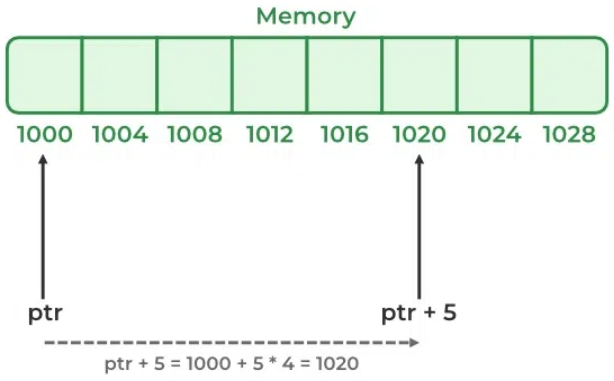
3.1.4 Uses and Advantages of Pointers

- *Dynamic Memory Allocation:* Pointers are crucial for allocating memory dynamically at runtime using functions like malloc(), calloc(), and realloc().
- *Passing Arguments by Reference:* Pointers allow functions to modify the original values of variables passed as arguments, rather than just working with copies.
- *Arrays and Strings:* Pointers provide an alternative and often more efficient way to access and manipulate elements in arrays and strings.
- *Data Structures:* Pointers are fundamental for building complex data structures like linked lists, trees, and graphs.
- *Improved Performance:* Direct memory access through pointers can sometimes lead to more optimized and faster code execution.

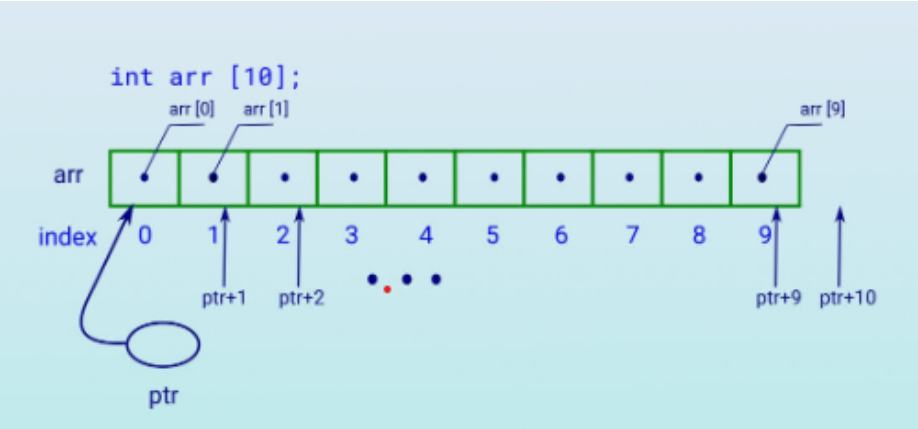
3.2 Pointer Arithmetic

- Pointer Arithmetic is the set of valid arithmetic operations that can be performed on pointers.
- These operations are:
    1. Increment/Decrement of a Pointer
    2. Addition of integer to a pointer
    3. Subtraction of integer to a pointer
    4. Subtracting two pointers of the same type

- **Increment (++) / Decrement (- -) of a Pointer**
    - When a pointer is incremented, it actually increments by the number equal to the size of the data type for which it is a pointer.
    - For Example: If an integer pointer that stores address 1000 is incremented, then it will increment by 4(size of an int), and the new address will point to 1004. While if a float type pointer is incremented then it will increment by 4(size of a float) and the new address will be 1004.
    - When a pointer is decremented, it actually decrements by the number equal to the size of the data type for which it is a pointer.
    - For Example: If an integer pointer that stores address 1000 is decremented, then it will decrement by 4(size of an int), and the new address will point to 996. While if a float type pointer is decremented then it will decrement by 4(size of a float) and the new address will be 996.
    - ptr++ → moves to the next element of the same data type.
    - ptr -- → moves to the previous element.
- **Addition of Integer to Pointer**
    - When a pointer is added with an integer value, the value is first multiplied by the size of the data type and then added to the pointer.
    - For Example: Consider the same example as above where the ptr is an integer pointer that stores 1000 as an address. If we add integer 5 to it using the expression, ptr = ptr + 5, then, the final address stored in the ptr will be ptr = 1000 + sizeof(int) * 5 = 1020.
    - p + n → moves forward n elements.



(a)



(b)

Fig. 3.2 Pointer Arithmetic

- **Subtraction of Integer to Pointer**
  - When a pointer is subtracted with an integer value, the value is first multiplied by the size of the data type and then subtracted from the pointer similar to addition.
  - For Example: Consider the same example as above where the ptr is an integer pointer that stores 1000 as an address. If we subtract integer 5 from it using the expression, ptr = ptr - 5, then, the final address stored in the ptr will be ptr = 1000 - sizeof(int) * 5 = 980.

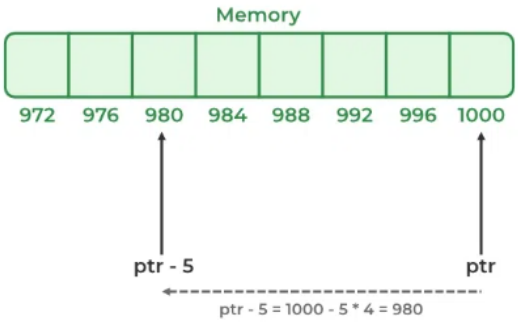


Fig. 3.2 Pointer subtraction

- **Subtraction of Two Pointers**
  - The subtraction of two pointers is possible only when they have the same data type (if both point to the same array).
  - If you have two pointers of the same type pointing into (or just past) elements of the same array, you can subtract one from the other.
  - ptr1 - ptr2: When two pointers of the same type are subtracted, the result is an integer representing the number of elements between the two memory locations.
  - Example:

```
int arr[] = {10, 20, 30, 40, 50};
```

```
int *p1 = &arr[1]; // points to 20
int *p2 = &arr[4]; // points to 50

int diff = p2 - p1; // number of int elements apart
printf("p2 - p1 = %d elements\n", diff); // prints 3
```

3.3 Passing Pointers to Functions

- When you pass a variable to a function by value, the function gets a copy. Any changes made inside the function won't affect the original variable.
- But if you pass a pointer (i.e., the address of a variable), the function can directly access and modify the original variable's value.
- Basic Syntax:

```
// In function definition
    void functionName(type *ptr) {
       // use *ptr to access or modify the original variable
    }
//Function call
    functionName(&variable); // pass address or call by reference
```

- Example: Passing pointer to modify a single variable

```
#include <stdio.h>
void increment(int *x) {
   *x = *x + 1;  // change original value
}

void main() {
   int num = 10;
   printf("Before increment: %d\n", num);
   increment(&num);    // pass address of num
   printf("After increment: %d\n", num);
}
```

Output:

Before increment: 10

After increment: 11

3.3.1 Advantages of Passing Pointer to Functions

- *Modification of Original Variables (Pass by Reference):* When a pointer to a variable is passed to a function, the function receives the memory address of the

original variable. This allows the function to directly access and modify the value of the original variable, effectively implementing "pass by reference."

- *Efficiency with Large Data Structures:* Passing large data structures (like arrays or structs) by value involves copying the entire structure, which can be computationally expensive and consume significant memory. Passing a pointer to the structure, however, only involves copying the memory address, leading to improved performance and reduced memory overhead, especially for large data.
- *Returning Multiple Values:* Functions typically return a single value. By passing pointers to variables as arguments, a function can modify multiple variables directly, thus effectively "returning" multiple values through these modified parameters.
- *Dynamic Memory Management:* Pointers are fundamental for dynamic memory allocation. Functions can allocate memory dynamically and return a pointer to that newly allocated memory, or receive pointers to dynamically allocated memory to manage its deallocation.
- *Working with Arrays and Strings:* Arrays in C are often treated as pointers to their first element. Passing arrays to functions is typically done by passing a pointer to the first element, allowing efficient access and manipulation of the entire array within the function. Similarly, string manipulation functions often operate on pointers to character arrays.

3.4 Arrays and Pointers

- In C, the name of an array is actually a pointer to the first element of the array.
- That is, an array name acts like a pointer to its first element.
- Example:

```
int a[5] = {10, 20, 30, 40, 50};
```

- a is the base address (address of a[0]).
- &a[0] = = a
- Pointer to first element:

```
int *p = a;      // same as p = &a[0];
```

3.4.1 Accessing Array Elements Through Pointers

- If:

```
int a[5] = {10, 20, 30, 40, 50};
int *p = a;
```

Then:

```
p[0] == a[0] == *p          // 10
p[1] == a[1] == *(p + 1)    // 20
```

You can move the pointer:

```
p = p + 2;  // now points to a[2]
printf("%d", *p); // 30
```

- Example:Program to traverse the array elements:

```
#include <stdio.h>
void main() {
   int arr[5] = {10, 20, 30, 40, 50};  // array
   int *p;                             // pointer

   p = arr;  // pointer points to first element of array (same as &arr[0])

   printf("Array elements are:\n");
   for (int i = 0; i < 5; i++) {
      printf("%d ", *(p + i)); // access element using pointer arithmetic
   }
 }
```

Output:
10 20 30 40 50

3.4.2 Passing Arrays to Functions

- Arrays automatically decay to pointers when passed to functions.
- Example: Program to traverse and print array elements

```
#include <stdio.h>
void printArray(int *arr, int size) {
   for(int i = 0; i < size; i++)
      printf("%d ",*(arr + i));
   printf("\n");
}
void main() {
   int numbers[5] = {1, 2, 3, 4, 5};
   printArray(numbers, 5);  // array decays to pointer
}
```

Output:

1 2 3 4 5

3.5 Strings and Pointers

- A string is a sequence of characters ending with a null character '\0'.
- A string can be declared mainly in two ways:
    - Using a character array as follows:

      ```
      char str[10] = "C Program";
      ```

      Here memory for 10 characters is allocated (including '\0').

    - Using a pointer:

      ```
      char *str = "C Program";
      ```

      Here, str points to a string literal stored in read-only memory.

- A string name (like str) acts as a pointer to its first character.
- Pointer arithmetic can be used to traverse or manipulate a string.
- Example:

```
void main() {
   char str[] = "Hello";
   char *ptr = str;   // pointer points to first element of str
   while (*ptr != '\0') {
      printf("%c ", *ptr);  // print character
      ptr++;              // move to next character
   }
  }
```

Output: H e l l o

- Program: Find Length of String Using Pointer

```
#include <stdio.h>
void main() {
   char str[100];
   char *ptr;
   int length = 0;
   printf("Enter a string: ");
   gets(str);
   ptr = str;  // pointer points to the first character of string
   while (*ptr != '\0') {  // loop until null character is found
      length++;
      ptr++;  // move to next character
   }
   printf("Length of the string = %d\n", length);

  }
```

3.6 Array of Pointers

- An array of pointers is an array in which each element is a pointer that can store the address of another variable (like int, float, char, or even a string).
- Syntax:

```
data_type *array_name[size];
```

- Example:

```
int *a[3];
```

- Program : Array of integer pointers

```
#include <stdio.h>
int main() {
   int a = 10, b = 20, c = 30;
   int *arr[3];     // array of 3 integer pointers
   arr[0] = &a;
   arr[1] = &b;
   arr[2] = &c;
   for (int i = 0; i < 3; i++) {
      printf("Value of arr[%d] = %d\n", i, *arr[i]);
   }
   return 0;
}
```

- Output

```
Value of arr[0] = 10
Value of arr[1] = 20
Value of arr[2] = 30
```

3.7 Dynamic Memory Allocation

- Dynamic memory allocation means allocating memory at runtime, rather than at compile time.
- To allocate memory dynamically, the following library functions are used:
  - malloc()
  - calloc()
  - realloc()
  - free()
- These functions are defined in the <stdlib.h>header file.

3.7.1 malloc()

- The malloc() (stands for memory allocation) function is used to allocate a single block of contiguous memory.

- The memory allocated by malloc() is uninitialized, that is, it contains garbage values.
- Syntax:
  ptr = (castType *)malloc(size_in_bytes);
- Example:
  int *ptr = (int *)malloc(5 * sizeof(int));
  It allocates memory for 5 integers .

3.7.2 calloc()

- The calloc() (stands for contiguous allocation) function is similar to malloc(), but it initializes the allocated memory to zero.
- It is used when you need memory with default zero values.
- Syntax:
  ptr = (castType *)calloc(num_of_blocks, size_of_each_block);
- Example:
  int *ptr = (int *)calloc(5, sizeof(int));

  It allocates memory for 5 integers and initializes all to 0.

3.7.3 realloc()

- The realloc() function is used to resize a previously allocated memory block.
- It allows you to change the size of an existing memory allocation without needing to free the old memory and allocate a new block.
- Syntax:
  ptr = (castType *)realloc(ptr, new_size_in_bytes);
- Example:
  ptr = (int *)realloc(ptr, 10 * sizeof(int));

  It resizes the existing allocated memory (from 5 integers to 10 integers).

3.7.4 free()

- The free() function is used to release dynamically allocated memory back to the operating system.
- It is essential to free memory that is no longer needed to avoid memory leaks.
- Syntax:
  free(ptr);
- Example:
  free(ptr);

  Releases the dynamically allocated memory back to the system.